

Gradient computation for the inverse problem: a comparison of differentiation methods

Sipan Bareyan

June 16, 2026

Abstract

We compare methods for computing the gradient $g = \nabla_V L$ of the misfit $L(V) = \int_0^{2\pi} (u(V) - u_0)^2 dx$, where the state $u(V)$ solves the discretized nonlinear equation $F(u, V) = 0$ by Newton’s method. The state solve and the adjoint construction are implemented in the library; we study only the *compute cost*, *memory*, and *accuracy* of the gradient, and how each scales with the number of parameters N . We contrast finite differences, automatic differentiation of the loss (forward and reverse), the hand-written adjoint, and AD equipped with a rule with different AD libraries in Julia.

1 Methods

The gradient maps the N entries of the discretized potential V to a scalar, so the question that matters is how its cost grows with N .

- **Finite differences (FD).** Perturb each entry in turn: $N+1$ state solves, accuracy limited to $\sqrt{\varepsilon_{\text{mach}}}$ by the step-size trade-off. Baseline.
- **Forward-mode AD.** One tangent direction per pass, so N passes. It propagates a dual number through the Newton iteration rather than unrolling it to scalars, so a pass costs a constant times one state solve: machine-accurate, but $\mathcal{O}(N)$ like FD.
- **Reverse-mode AD.** A single backward pass returns the whole gradient, but it traces the Newton iteration and stores every intermediate state, so memory grows with the iteration count.
- **Adjoint (analytic).** One extra linear solve with the Newton-step operator at the converged solution: cost independent of N and comparable to a single state solve. This is the reference.
- **Rule-based AD.** A custom rule for the state solve makes the AD tool compose the analytic adjoint instead of differentiating the iteration. It runs on the production solver and stays differentiable inside a larger pipeline.

Differentiating the linear solve. Every reverse-mode path carries a custom rule for the inner conjugate-gradient solve, so the *linear* system is never differentiated through; AD sees only the *Newton* iteration. (Forward mode simply propagates duals through the fixed-iteration solver.) The naive-versus-ruled comparison then measures the cost of unrolling Newton against the cost of replacing it by the adjoint rule, not the easily avoided cost of unrolling CG.

Table 1: Cost of one gradient evaluation. $T_{\text{sol}} = n_N T_{\text{lin}}$ is one state solve, $T_{\text{lin}} = \mathcal{O}(n_c N \log N)$ one linear solve.

Method	State solves	Compute	Memory	Accuracy
Finite differences	$N+1$	$\mathcal{O}(N T_{\text{sol}})$	$\mathcal{O}(N)$	$\mathcal{O}(\sqrt{\varepsilon_{\text{mach}}})$
Forward AD (duals)	$\sim N$	$\mathcal{O}(N T_{\text{sol}})$	$\mathcal{O}(N)$	machine, at iterate
Reverse AD naive (CG-ruled)	1+ bwd	$\mathcal{O}(T_{\text{sol}})$	$\mathcal{O}(n_N N)$	machine, at iterate
Adjoint	1	$\mathcal{O}(T_{\text{sol}})$	$\mathcal{O}(N)$	exact, at u^*
Rule-based AD	1	$\mathcal{O}(T_{\text{sol}})$	$\mathcal{O}(N)$	= adjoint

2 Theoretical complexity

Let N be the number of parameters (grid points), n_N the number of Newton iterations, and n_c the number of CG iterations per linear solve. One spectral operator application costs $\mathcal{O}(N \log N)$, so one preconditioned CG solve costs $T_{\text{lin}} = \mathcal{O}(n_c N \log N)$ and one state (Newton) solve costs $T_{\text{sol}} = n_N T_{\text{lin}}$. We express each gradient cost in these units; table 1 summarizes.

Two regimes. FD and forward-mode AD cost $\mathcal{O}(N)$ state solves — the price of probing one input direction at a time. Reverse-mode AD, the adjoint and rule-based AD return the *entire* gradient at $\mathcal{O}(1)$ state solves, comparable to a single loss evaluation and independent of N : the classical reverse-mode result [1, 2].

Within the cheap group. What separates the $\mathcal{O}(1)$ -solve methods is the constant and the memory. The hand adjoint adds one linear solve and stores nothing extra. Naive reverse-mode redoes the iteration backward — $\approx n_N$ extra linear solves ($\approx 2\times$ the adjoint) and a tape of the n_N Newton iterates ($\mathcal{O}(n_N N)$ memory). Rule-based AD injects exactly that adjoint, recovering the single-solve cost and $\mathcal{O}(N)$ memory while staying composable. On accuracy, forward- and reverse-mode AD differentiate the program — the n_N -th iterate $u_{n_N}(V)$ — returning the exact derivative of an approximate solution, off by $\mathcal{O}(\text{Newton residual})$; the adjoint and the state-solve rule differentiate the converged solution through one exact linear solve, matching to the linear-solve tolerance.

3 Protocol

We fix a well-conditioned instance — a forcing with nonzero mean, so the state stays bounded away from zero — and a single evaluation point V_0 . Before any timing, every gradient is checked against finite differences, so a method that is fast only because it is wrong is excluded. *Time* is the minimum over warmed-up samples (BenchmarkTools, JIT excluded); *memory* is the allocation per call; *accuracy* is the relative ℓ_2 deviation from the adjoint gradient. Times are also read in units of one state solve, the natural unit of table 1. The study runs over $N = 5, \dots, 500$. The AD tools are ForwardDiff (forward mode) and Mooncake and Zygote (reverse mode); the reverse-mode paths use the CG rule, and the rule-based paths add the state-solve rule.

4 Results

Accuracy. Table 2 and fig. 1 report the relative ℓ_2 deviation of each gradient from the analytic adjoint reference at $N = 500$.

[State the FD deviation, the naive-AD deviation (the iterate gap, of order the Newton residual), and confirm that rule-based AD sits at the adjoint level rather than the iterate level.]

Table 2: Relative ℓ_2 deviation of each gradient from the analytic adjoint reference.

category	tool	mode	rel_err
<i>String</i>	<i>String</i>	<i>String</i>	<i>Float64</i>
FD	-	-	4.42725e-6
adjoint	-	reverse	0.0
naive	ForwardDiff	forward	6.87036e-6
naive	Mooncake	reverse	1.97759e-6
naive	Zygote	reverse	1.97759e-6
ruled	Mooncake	reverse	5.57196e-12
ruled	Zygote	reverse	5.57196e-12

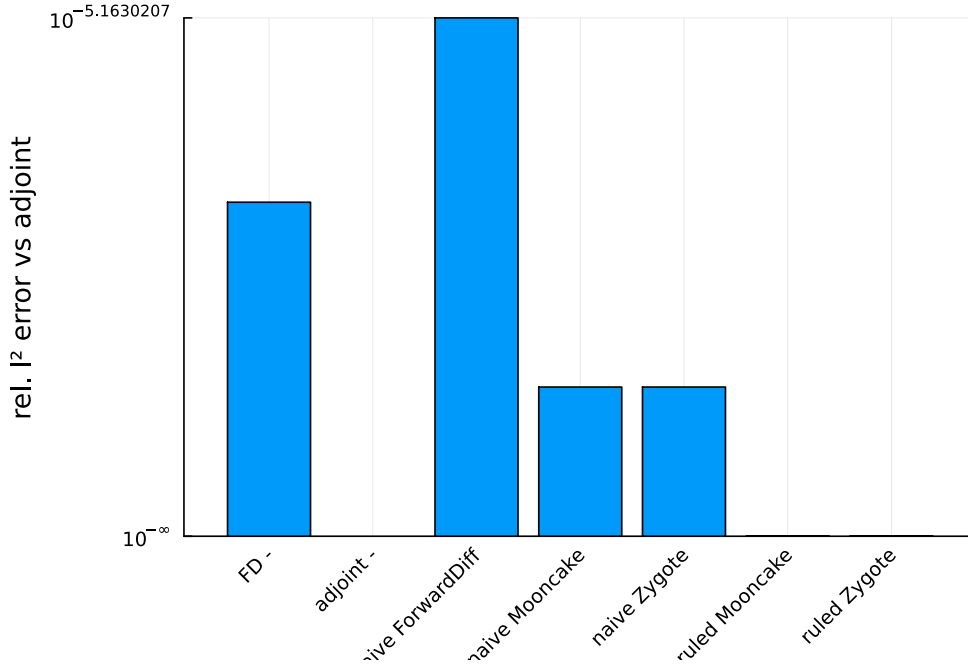


Figure 1: Relative ℓ_2 error of each method against the analytic adjoint (log scale).

Cost at fixed size. Table 3 gives wall time and allocations at $N = 500$, with each time also as a multiple of one state solve.

[Quote the adjoint and rule-based times in units of one solve ($\approx 1-2$), the FD and forward-AD slowdown ($\sim N$ solves), and the reverse-naive allocation relative to the adjoint, where the $\mathcal{O}(n_N N)$ tape shows up even when the time is competitive.]

Scaling in N . Figures 2 and 3 show time and memory against N on log-log axes; the per- N tables below give the backing data, one table per grid size, each reporting the relative ℓ_2 error against the analytic adjoint.

[Read off the slopes: FD and forward-AD have unit slope in time ($\mathcal{O}(N)$ solves); reverse-mode, adjoint and rule-based are flat. In memory, reverse-naive grows with N while the others stay flat. Give the crossover N beyond which the adjoint is decisively cheaper than FD.]

5 Discussion

The measurements confirm table 1. Finite differences and forward-mode AD scale as $\mathcal{O}(N)$ state solves and are usable only as baselines. Reverse-mode AD, the adjoint, and rule-based AD all

Table 3: Time and allocations per gradient evaluation at fixed N .

category	tool	mode	N	time_s	alloc_bytes	rel_err
<i>String</i>	<i>String</i>	<i>String</i>	<i>Int64</i>	<i>Float64</i>	<i>Int64</i>	<i>Float64</i>
FD	-	-	500	5.29439	13370617208	4.42725e-6
adjoint	-	reverse	500	0.00791274	27612248	0.0
naive	ForwardDiff	forward	500	86.2609	128850421472	6.87036e-6
naive	Mooncake	reverse	500	11.7867	20974668224	1.97759e-6
naive	Zygote	reverse	500	0.252095	1618399336	1.97759e-6
ruled	Mooncake	reverse	500	0.015671	54230416	5.57196e-12
ruled	Zygote	reverse	500	0.00783365	27459376	5.57196e-12

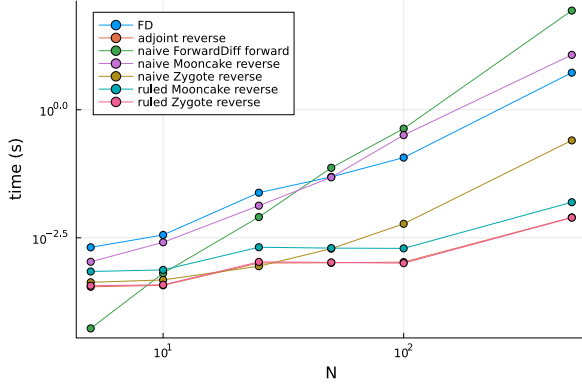


Figure 2: Time versus N .

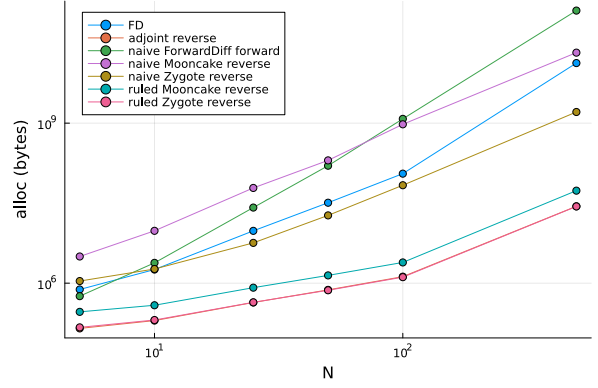


Figure 3: Allocations versus N .

compute the full gradient at the cost of $\mathcal{O}(1)$ solves; within that group the hand adjoint is the cheapest and lightest, naive reverse-mode pays the Newton-unrolling factor in both compute ($\approx 2\times$) and memory ($\mathcal{O}(n_N N)$), and rule-based AD recovers the adjoint's efficiency while remaining composable. For production the hand adjoint is the clear choice; rule-based AD is preferable when the gradient must chain with other differentiated code; naive differentiation is kept only as a baseline and a correctness cross-check.

Reproducibility. Julia 1.12.6, BLAS threads 8; commit 47606ff, 2026-06-16.

References

- [1] S. G. Johnson, *Notes on Adjoint Methods for 18.335*, MIT, 2006 (updated 2021).
- [2] A. Levitt, *The Adjoint Trick*, 2020.

Table 4: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 5$).

category <i>String</i>	tool <i>String</i>	mode <i>String</i>	time_s <i>Float64</i>	alloc_bytes <i>Int64</i>	rel_err <i>Float64</i>
FD	-	-	0.00206272	761280	1.15524e-7
adjoint	-	reverse	0.000349728	141968	0.0
naive	ForwardDiff	forward	5.3978e-5	570864	1.51471e-5
naive	Mooncake	reverse	0.00107801	3163848	5.26179e-8
naive	Zygote	reverse	0.000428474	1095872	5.26179e-8
ruled	Mooncake	reverse	0.000696563	289904	3.50732e-16
ruled	Zygote	reverse	0.000367961	148384	3.50732e-16

Table 5: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 10$).

category <i>String</i>	tool <i>String</i>	mode <i>String</i>	time_s <i>Float64</i>	alloc_bytes <i>Int64</i>	rel_err <i>Float64</i>
FD	-	-	0.00359994	1801072	1.8179e-7
adjoint	-	reverse	0.000374005	198912	0.0
naive	ForwardDiff	forward	0.000643048	2403760	1.64467e-5
naive	Mooncake	reverse	0.00258311	9582984	1.22867e-7
naive	Zygote	reverse	0.000476631	1842800	1.22867e-7
ruled	Mooncake	reverse	0.000747604	385504	3.53916e-16
ruled	Zygote	reverse	0.000382403	203792	3.53916e-16

Table 6: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 25$).

category <i>String</i>	tool <i>String</i>	mode <i>String</i>	time_s <i>Float64</i>	alloc_bytes <i>Int64</i>	rel_err <i>Float64</i>
FD	-	-	0.0240583	9567904	3.42596e-7
adjoint	-	reverse	0.00101917	434576	0.0
naive	ForwardDiff	forward	0.00804734	26142800	1.26784e-5
naive	Mooncake	reverse	0.0134243	60995584	1.33311e-7
naive	Zygote	reverse	0.000881939	5693792	1.33311e-7
ruled	Mooncake	reverse	0.00207198	823232	2.98367e-16
ruled	Zygote	reverse	0.00107246	437152	2.98367e-16

Table 7: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 50$).

category <i>String</i>	tool <i>String</i>	mode <i>String</i>	time_s <i>Float64</i>	alloc_bytes <i>Int64</i>	rel_err <i>Float64</i>
FD	-	-	0.0489131	32215600	5.58154e-7
adjoint	-	reverse	0.00102984	748624	0.0
naive	ForwardDiff	forward	0.0736605	159223304	1.26773e-5
naive	Mooncake	reverse	0.0479491	200426728	1.96469e-7
naive	Zygote	reverse	0.001944	18648224	1.96469e-7
ruled	Mooncake	reverse	0.00199203	1398688	5.57772e-12
ruled	Zygote	reverse	0.00104815	736256	5.57772e-12

Table 8: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 100$).

category <i>String</i>	tool <i>String</i>	mode <i>String</i>	time_s <i>Float64</i>	alloc_bytes <i>Int64</i>	rel_err <i>Float64</i>
FD	-	-	0.116405	113004928	6.72626e-7
adjoint	-	reverse	0.00106435	1324416	0.0
naive	ForwardDiff	forward	0.428056	1202579944	1.2677e-5
naive	Mooncake	reverse	0.320082	951243992	1.9828e-6
naive	Zygote	reverse	0.00594222	68566784	1.9828e-6
ruled	Mooncake	reverse	0.00196825	2447400	5.57336e-12
ruled	Zygote	reverse	0.00101235	1295856	5.57336e-12

Table 9: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 500$).

category <i>String</i>	tool <i>String</i>	mode <i>String</i>	time_s <i>Float64</i>	alloc_bytes <i>Int64</i>	rel_err <i>Float64</i>
FD	-	-	5.29439	13370617208	4.42725e-6
adjoint	-	reverse	0.00791274	27612248	0.0
naive	ForwardDiff	forward	86.2609	128850421472	6.87036e-6
naive	Mooncake	reverse	11.7867	20974668224	1.97759e-6
naive	Zygote	reverse	0.252095	1618399336	1.97759e-6
ruled	Mooncake	reverse	0.015671	54230416	5.57196e-12
ruled	Zygote	reverse	0.00783365	27459376	5.57196e-12